

UVa12164/LA4412 The Great Game

题目链接

本题是[2008年ICPC 亚洲区域赛 吉隆坡赛区](#)的题目

题意

石头剪刀布的游戏规则是这样的：两个人一起出拳，必须出石头、剪刀、布之一。石头胜剪刀，剪刀胜布，布胜石头。你和某人玩石头剪刀布游戏，分为若干轮，每轮出 G ($1 \leq G \leq 1000$) 次拳。胜者得1分（如果两个人出的一样，都不得分）。每轮结束后，得分多的胜出（如果两人得分相同，则该轮没有人胜出）。当你的对手比你多赢 L 轮时，你就算输掉了整个比赛；当你比对手多赢 W 轮时，你就算赢得了整个比赛。你的任务是找一个最优策略，使赢得整个比赛的概率最大。 $1 \leq W, L \leq 100$ 。假定你的对手的策略是固定的，而且每轮都一样：第 i 次出拳时分别有 $ai\%$ ， $bi\%$ ， $ci\%$ 的概率出石头、布和剪刀。输入保证 $ai + bi + ci = 100$

分析

非常好的一道难题，综合了马尔可夫链，二分，dp。在我之前，[uhunt](#)上查到仅[dreamoon](#)(台湾知名ACM选手)、[josh bao](#)、[try](#)三位神仙通过了，并且看不到代码。紫书的题解有两个说得不明确的地方，我想了很久才弄明白。

紫书题解

你的任务是比对手多赢 W 轮，而各轮之间是不相关的，所以你需要每一轮都玩得尽量好。可是什么叫“玩得尽量好”呢？如果每一轮只有赢和输两种可能，那么“玩得尽量好”就是指获胜的概率尽量大。但是在本题中，每一轮除了输赢之外还有可能是平局。如果有两种策略，一种是20%概率赢，80%概率平（因此不可能输），但另外一种80%概率赢，10%概率平（因此还有10%的概率输），哪种策略更好呢？仔细思考后会发现：虽然第一种策略的胜率比较低，但它是必胜的（即答案是100%）——对手没有任何机会获胜；第二种策略虽然赢的概率比较大，但却有概率输掉，如果 $L = 1$ ，答案肯定不是100%。

《训练指南》中曾经介绍过马尔科夫链。如果用一个编号为 x 的结点表示“比对手多赢 x 场”这个状态，则本题就是一个包含 $L + W + 1$ 个结点（即 $-L, -(L-1), \dots, 0, 1, \dots, W$ ）的马尔科夫链，要求一个策略使得结点0直达结点 W 的概率最大。

假设最优策略使得每局获胜的概率为 P_{win} ，输掉的概率为 P_{lose} ，每个内结点（即不是 $-L$ 也不是 W 的结点）往左的转移概率为 P_{lose} ，往右转移的概率为 P_{win} ，转移到自己的概率为 $1 - P_{lose} - P_{win}$ 。因为本题并不关心到达结点 $-L$ 或 W 的具体时间，只关心先到达 W 的概率，所以刚才的马尔科夫链等价于去掉自环（即每个状态到自身的转移），然后把往左往右的概率归一化（即让二者加起来

等于1)。此处要最大化的正是这条新马尔科夫链中的获胜 概率，即 $P_0 = P_{win} / (P_{win} + P_{lose})$ 。

至此，问题分成了两个完全独立的部分：如何最大化 P_0 ，以及已知 P_0 之后如何求出状态W的首达概率。后者的一般做法如下：设状态i时的获胜概率为d(i)，根据边界 $d(-L) = 0$ ， $d(W) = 1$ 以及马尔科夫方程联立求解。具体解法在《训练指南》中已有详细叙述。对于本题中特殊的马尔科夫链，还可以直接求出解的封闭形式。另外，还可以用迭代法而非高斯消元法解方程组，这里不再详述。

前者也有两种解法：二分法和不动点迭代法。不动点迭代法及其收敛性的证明超出了本书的讨论范围，因此这里只介绍二分法。二分答案p，看看是否有一种策略使得 $P_{win} / (P_{win} + P_{lose}) \geq p$ ，即 $(1-p) * P_{win} - p * P_{lose} \geq 0$ 。接下来就只需动态规划计算 $(1-p) * P_{win} - p * P_{lose}$ 的最大值了。令“胜”的权值为 $1-p$ ，“负”的权值为 $-p$ ，则问题转化为最大化权值的数学期望。设状态 $d(i, j)$ 表示前i次猜拳，得分为j（注意j可能为负数）时的最大期望，分剪刀、石头、布3种情况讨论即可。

我主要是对题解中二分答案p之后的dp细节（令“胜”的权值为 $1-p$ ，“负”的权值为 $-p$ ）有些迷糊，后面加入两点个人理解后ac了：1、“平”的权值为0；2、胜、负、平说的都是针对一轮最终结果的权重而非本轮一次猜拳结果的权重。

也就是说边界条件为：

$$d(g,1) = d(g,2) = \dots = d(g,g) = 1-p$$

$$d(g,-1) = d(g,-2) = \dots = d(g,-g) = -p$$

$$d(g,0) = 0$$

AC代码

```
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;

#define G 1020
int g, w, l; double d[G][G<<1], p[G][3];

bool check(double qw, double ql) {
    d[g][0] = d[g][g+1] = 0.;
    for (int i=1; i<=g; ++i) d[g][i] = qw, d[g][i+g+1] = ql;
    for (int i=g-1; i>=0; --i) {
```

```

double e0 = p[i][2]*d[i+1][1] + p[i][0]*d[i+1][0] + p[i][1]*d[i+1][g+2];
double e1 = p[i][0]*d[i+1][1] + p[i][1]*d[i+1][0] + p[i][2]*d[i+1][g+2];
double e2 = p[i][1]*d[i+1][1] + p[i][2]*d[i+1][0] + p[i][0]*d[i+1][g+2];
d[i][0] = d[i][g+1] = max(max(e0, e1), e2);
for (int j=1; j<=i; ++j) {
    e0 = p[i][2]*d[i+1][j+1] + p[i][0]*d[i+1][j] + p[i][1]*d[i+1][j-1];
    e1 = p[i][0]*d[i+1][j+1] + p[i][1]*d[i+1][j] + p[i][2]*d[i+1][j-1];
    e2 = p[i][1]*d[i+1][j+1] + p[i][2]*d[i+1][j] + p[i][0]*d[i+1][j-1];
    d[i][j] = max(max(e0, e1), e2);
    e0 = p[i][2]*d[i+1][j+g] + p[i][0]*d[i+1][j+g+1] + p[i][1]*d[i+1][j+g+2];
    e1 = p[i][0]*d[i+1][j+g] + p[i][1]*d[i+1][j+g+1] + p[i][2]*d[i+1][j+g+2];
    e2 = p[i][1]*d[i+1][j+g] + p[i][2]*d[i+1][j+g+1] + p[i][0]*d[i+1][j+g+2];
    d[i][j+g+1] = max(max(e0, e1), e2);
}
}
return d[0][0] > 0.;
}

double solve() {
    for (int i=0; i<g; ++i) {
        cin >> p[i][0] >> p[i][1] >> p[i][2];
        p[i][0] /= 100.; p[i][1] /= 100.; p[i][2] /= 100.;
    }
    double low = 0., high = 1.;
    while (low + 1e-8 <= high) {
        double mid = (low + high) / 2;
        check(1-mid, -mid) ? low = mid : high = mid;
    }
    if (high == .5) return 1.*l/(w+l);
    double k = (1.-high) / high;
    return (1.-pow(k, l)) / (1. - pow(k, w+l));
}

int main() {
    cout << fixed << setprecision(3);
    while (cin>>g>>w>>l && g) cout << 100*solve() << '%' << endl;
    return 0;
}

```